

青少年 C++ 算法可视化教学系统

让抽象算法看得见、写得出、练得会

可运行全栈 MVP | 算法动画 × 在线判题 × 学习数据闭环



图 1 课程页将算法动画、理解路径、C++ 代码与练习入口放在同一学习空间

算法章节	课程课时	唯一题目	动画课时
9	84	201	35

一句话价值 用可复用的算法动画把抽象过程讲清楚，用在线判题把“看懂”转化为“会写”，再用学习数据帮助教师把下一次课讲得更准。

一、评委导读：为什么值得进入下一轮

本项目面向青少年 C++ 与信息学竞赛入门场景，聚焦两个长期痛点：一是算法中的状态、指针、调用栈和边界过程难以仅靠板书讲清；二是高质量动画课程制作成本高、练习与教学反馈彼此割裂。项目已经实现 React 学习前台、Manim 动画生产、FastAPI 判题服务与 SQLite 数据看板，形成可本地运行、可展示、可继续扩展的完整 MVP。

建议评委先看 项目的核心创新不在某一张动画或某一道题，而在同一知识点从动画、代码、练习、判题到教师诊断的全链路一致映射。

评委关注	常见单点方案	本项目的回答
是否真能学会	只看动画或只刷题，理解与编码脱节	动画步骤、C++ 变量、练习测试点使用一致知识映射
是否可规模化	每个知识点重复手工制作	统一 Manim 主题、组件、脚本和课时数据约定
是否能迭代	课程发布后缺少反馈证据	提交、通过率、薄弱题与错误类型回流教师看板

项目落地佐证

1. 当前课程数据实际包含 9 个章节、84 个课时、203 个课时练习引用（去重后 201 个唯一题目）；35 个课时已接入 Manim 动画。
2. 7 个章节状态为 ready，2 个章节处于持续建设；高精度、排序、递推与递归已形成动画课时样板。
3. 在线 C++17 编辑、自动保存、提交、测试点结果、提交历史、进度联动与教师统计看板均可运行。
4. Judge Adapter 已抽象 LocalJudge / DockerJudge，两类判题实现可替换，便于从本地演示过渡到隔离部署。

申报结论

项目已越过“只有创意或界面原型”的早期阶段：内容、交互、判题、数据和演示脚本均有代码与界面证据。参赛价值在于把教学创新与工程落地放在同一套系统中验证，并为规模化内容生产留下清晰接口。

二、核心创新：把单点工具连成学习闭环



图 2 动画理解—代码映射—在线练习—即时判题—分层反馈—教师诊断

创新 1 | 动画不是终点，而是代码理解的入口。

动画中的数组格、指针、调用栈、状态转移与递归树继续映射到 C++ 变量、循环边界与测试点，减少“动画看懂了，代码仍不会写”的断层。

创新 2 | 分层反馈替代直接给答案。

练习先提示思路，再提示关键变量与边界，最后才进入核心实现；同时提供题解要点与常见错误，为青少年提供适度支架。

创新 3 | 教师看板把错误转化为教学动作。

系统聚合通过率、薄弱题、错误类型与平均尝试次数，把学生提交转化为下一次课的复讲重点与练习梯度建议。

创新 4 | 内容生产流程可复用。

统一 Manim 主题、数组组件、预览图、场景命名与逐镜头脚本，让新增知识点能够复用已有视觉语言和工程流程。

评委看点 创新点均有对应运行证据：图 4 证明学生端闭环，图 5 证明在线判题，图 6 证明数据回流；并非仅停留在概念描述。

三、系统架构与工程创新



图 3 学习交互层、课程内容层、服务判题层与数据反馈层协同

模块化设计带来的可扩展性

- 5. **前端**：React + TypeScript + Vite 负责课程导航、进度、视频、讲义、练习与教师看板。
- 6. **内容**：Manim 负责算法动画生产，课程数据模型统一承载课时、示例、练习与媒体路径。
- 7. **服务**：FastAPI 提供提交、判题、历史、统计与能力查询接口。
- 8. **判题**：JudgeService 将业务层与执行环境隔离，可在 LocalJudge、DockerJudge 或外部 OJ Adapter 之间切换。
- 9. **数据**：SQLite 记录提交并生成聚合统计，浏览器本地状态在后端不可用时提供体验兜底。

方案深度佐证

项目不是把多个开源组件简单拼接：课程内容模型决定动画与练习的映射，Judge Adapter 约束判题边界，统计看板又从提交记录计算教学指标。每一层都围绕同一教学闭环设计，因此能够在保持模块解耦的同时形成产品协同。

四、学生端证据 1：从动画到代码的同屏学习



图 4 高精度加法课程页：动画、概念标签、理解路径、C++ 示例与课时导航

界面如何服务学习，而不是只追求展示

- 10.动画区把“反向存储、逐位相加、carry 进位、取模与整除”拆成可观察步骤。
- 11.概念标签与理解路径帮助学生在观看过程中建立术语和步骤锚点。
- 12.C++ 示例与动画共用变量含义，学生可把画面中的变化直接对应到代码。
- 13.课时导航、标记完成、练习状态和继续学习入口共同维护学习节奏。

教学设计意图 让学生先形成过程模型，再进入语法实现；降低一上来面对整段代码时的认知负荷。

五、学生端证据 2：在线编码、判题与复盘



图 5 Monaco C++17 编辑器、判题结果与最近提交在同一页面完成

可验证的练习闭环

14. 学生从课程练习入口进入题目，编辑器自动加载起始代码并保存草稿。
15. 提交后由 FastAPI 调用 Judge Adapter，执行公开与隐藏测试点。
16. 页面展示 Accepted、Wrong Answer、Compile Error 等结果和通过测试点数量。
17. Accepted 自动计入学习进度；错误提交保留在最近记录中，便于回看和复盘。
18. 分层提示、题解要点和常见错误在卡住时提供逐步帮助，而不是直接暴露完整答案。

工程边界 当前 local-process 判题器仅用于 MVP 本地演示；正式部署需启用 Docker 沙箱或接入外部 OJ，项目已为此预留 Adapter。

六、教师端证据：从提交记录到教学诊断



图 6 提交统计、练习覆盖、薄弱题、错误类型和教师建议

评委一眼应看到的三个变化

- 19.从“学生做没做”升级为“学生卡在哪里”：薄弱题排行结合低通过率、无 Accepted 和错误提交密度。
 - 20.从“总体分数”升级为“错误类型”：答案错误、编译错误等分布帮助教师判断是概念、边界还是语法问题。
 - 21.从“展示数据”升级为“建议行动”：看板根据薄弱题、错误类型和练习覆盖给出下一次课建议。
- 当前截图使用可重复生成的 Demo 提交数据，用于稳定展示统计逻辑；后续接入真实账号、班级和日期筛选后，可直接扩展为课堂反馈看板。

七、课程规模与内容生产证据

下表由当前前端课程配置统计，数据对应仓库中的实际课时与练习，不把规划中的空壳能力计入已完成成果。

章	主题	状态	课时	练习	动画
1	高精度计算	ready	11	33	11
2	数据排序	ready	6	18	6
3	递推算法	building	9	27	9
4	递归算法	ready	9	27	9
5	搜索与回溯	building	10	20	0
6	贪心算法	ready	10	20	0
7	分治算法	ready	8	16	0
8	广度优先搜索	ready	9	18	0
9	动态规划	ready	12	24	0
合计		7 ready / 2 building	84	203	35

表 1 当前课程覆盖与动画接入情况（203 为课时引用数，去重后为 201 个唯一题目）

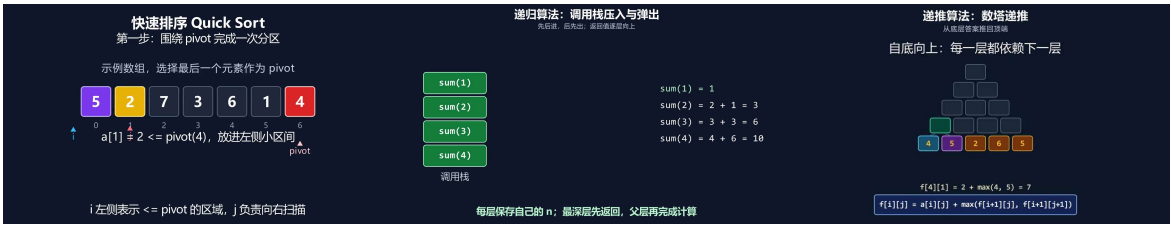


图 8 跨章节动画视觉语言：快速排序、递归调用栈与数塔递推

高精度、排序、递推和递归已形成统一视觉语言；搜索与回溯、贪心、分治、BFS 与动态规划已具备课时与练习结构，其中部分章节配有逐镜头制作脚本，动画将按现有模板持续补齐。

八、参赛优势、可行性与验证

相对优势

- 22.相对录播课程：学生不是被动观看，而是立即进入同知识点的代码与练习。
- 23.相对在线题库：系统先建立过程理解，再通过分层提示和题解完成反馈。
- 24.相对单个动画 Demo：项目有统一课程结构、201 个唯一题目、判题接口和数据回流。
- 25.相对概念型参赛方案：可一键启动，有真实页面、API、提交记录和演示检查脚本。

工程验证

验证维度	证据
前端	TypeScript 编译与 Vite 生产构建可执行；课程媒体路径与题目 ID 可检查。
后端	FastAPI 健康检查、提交、历史、统计和 Judge 能力查询接口已实现。
内容	Manim 场景提供 480p15 预览与多项 1080p60 成片，预览图直接进入课程。
演示	seed_demo_data.py 可重复生成看板数据，demo_start / stop / check 脚本降低现场风险。

预期价值

对学生，降低算法抽象门槛并缩短从理解到独立编码的距离；对教师，提供可观察的错误与复讲依据；对内容团队，降低跨章节动画与练习生产成本；对平台，提供可独立接入的课程、判题与统计模块。

九、风险控制、路线图与现场演示

已识别风险及控制措施

判题安全：本地进程判题仅用于演示；生产环境切换 Docker 沙箱或外部 OJ Adapter。

内容完整度：明确区分 35 个已接入动画课时与尚待成片章节，不把规划能力包装为完成成果。

数据真实性：Demo 数据只用于稳定展示统计逻辑；真实课堂评估将引入账号、班级与时间维度。

依赖与部署：演示前运行检查脚本，后续补充容器化、依赖锁定与持续集成。

下一阶段路线图

- 26. 补齐搜索与回溯、贪心、分治、BFS、动态规划的重点动画成片。
- 27. 完成 Docker 判题器实测、资源限制、危险系统调用控制与并发队列。
- 28. 接入真实学生与班级，增加日期、章节和学生维度筛选。
- 29. 开展课堂试点，以前后测、完成率、平均尝试次数和典型错误变化评估效果。
- 30. 在行为数据基础上生成个性化复习路径与教师课后复习包。

建议 3 分钟现场演示路径

- 31. 打开高精度加法课时：展示动画与变量映射。
- 32. 进入“大整数加法”：展示编辑、提交与测试点结果。
- 33. 返回课程：展示 Accepted 自动计入进度。
- 34. 打开教师看板：展示薄弱题、错误分布与教学建议。

最终结论 本项目以可运行工程证明了一条清晰路径：把算法过程可视化，把理解结果转化为代码能力，再把练习数据转化为教学决策。它既具备比赛现场的可展示性，也具备后续真实课堂验证和平台化扩展的基础。